

Infrastructure Orchestration & Automation

Bloc 3 — Implémenter et automatiser le déploiement des solutions de données

TP guidé — Jour 1

Construire un pipeline de données avec Apache Airflow

Enseignant	Dr. Abdelhak TOUITI
Durée	3h30 (après-midi)
Modalité	Individuel ou binôme
Compétences visées	C7.1, C7.3
Outil principal	Apache Airflow (Apache 2.0, open source)

Fait suite au cours du matin : enjeux de l'orchestration et de l'automatisation.

Présentation du TP

Ce matin, vous avez vu pourquoi automatiser un traitement de données réduit les risques et améliore la traçabilité. Cet après-midi, vous allez le mettre en pratique : construire un pipeline complet avec Apache Airflow, capable de gérer des dépendances entre tâches, de reprendre automatiquement après une erreur, et de conserver un historique exploitable de chaque exécution.

Astuce

Ce TP a été vérifié pas à pas sur **Apache Airflow 3.x** (le fichier Docker Compose “stable” pointe vers la dernière version majeure). Si l’interface que vous voyez diffère de captures d’écran trouvées en ligne datant d’Airflow 2, c’est normal : Airflow 3 a réorganisé une partie de la navigation (voir Étape 2). Le contenu ci-dessous reflète le comportement réel d’Airflow 3.

Objectifs pédagogiques

- Installer et démarrer Apache Airflow avec Docker Compose.
- Écrire un DAG Python avec plusieurs tâches dépendantes.
- Configurer la gestion des erreurs : *retries*, délai entre tentatives, notification d’échec.
- Vérifier la traçabilité des exécutions dans l’interface web.

Prérequis

- Docker et Docker Compose installés et fonctionnels (`docker --version`, `docker compose version`).
- Un éditeur de texte / IDE.
- Notions de base en Python (fonctions, imports).
- Au moins 4 Go de RAM disponibles pour les conteneurs Airflow.

Livrable attendu

- Le fichier `dags/pipeline_ventes.py` finalisé.
- Une capture d’écran de la page *Task Instances* d’une exécution réussie (onglet *Runs* → cliquer sur un run).
- Le contenu du fichier `notifications.log` généré lors d’un échec simulé.
- Les réponses aux questions de compréhension (section 6).

1 Installer Apache Airflow avec Docker Compose

Étape 1

Airflow fournit un fichier Docker Compose officiel qui démarre tous les composants nécessaires en une seule commande (scheduler, api server – l’interface web –, processeur de DAGs, worker, base de métadonnées...).

1.1 — Créer le dossier de travail

```
mkdir tp-airflow
cd tp-airflow
```

```
mkdir dags, logs, plugins, config
```

1.2 — Récupérer le fichier Docker Compose officiel

```
Invoke-WebRequest `
  -Uri "https://airflow.apache.org/docs/apache-airflow/stable/docker-compose.yaml" `
  -OutFile "docker-compose.yaml"
```

1.3 — Définir l'identifiant utilisateur

```
Set-Content -Path .env -Value "AIRFLOW_UID=50000"
```

Astuce

Sous Linux/macOS, la variable `AIRFLOW_UID` sert à faire correspondre les permissions de fichiers entre l'hôte et le conteneur ; ce concept n'existe pas sous Windows, d'où la valeur fixe 50000. Vérifiez aussi que Docker Desktop utilise bien le backend WSL2 (*Settings* → *General* → *Use the WSL 2 based engine*) : c'est la configuration recommandée pour Airflow sous Windows.

Équivalent bash (Linux/macOS), si un membre de votre binôme travaille sur ce type de poste plutôt que PowerShell :

```
mkdir -p tp-airflow/{dags,logs,plugins,config} && cd tp-airflow
curl -LfO \
  'https://airflow.apache.org/docs/apache-airflow/stable/docker-compose.yaml'
echo -e "AIRFLOW_UID=$(id -u)" > .env
```

1.4 — Initialiser puis démarrer Airflow

```
docker compose up airflow-init
docker compose up -d
```

Astuce

Un avertissement `"FERNET_KEY" variable is not set` peut s'afficher au démarrage : sans danger pour ce TP. Cette clé sert à chiffrer les données sensibles (mots de passe de connexions, variables marquées "sensibles") dans la base de métadonnées ; sans elle, elles sont simplement stockées en clair. Comme ce TP ne manipule aucun secret réel, vous pouvez ignorer l'avertissement.

Vérification

Ouvrez <http://localhost:8080> dans votre navigateur. Connectez-vous avec l'utilisateur `airflow` et le mot de passe `airflow` (identifiants par défaut du Docker Compose officiel). Vous devez voir la liste des DAGs d'exemple fournis par Airflow.

2 Découvrir l'interface Airflow

Étape 2

Avant d'écrire votre propre DAG, prenez cinq minutes pour explorer l'interface avec un DAG d'exemple (`example_bash_operator` par exemple).

Astuce

Dans Airflow 3, les vues *Grid* et *Graph* ne sont plus des onglets séparés au niveau du DAG. Elles vivent à l'intérieur d'une exécution précise (un *Dag Run*). Un DAG qui n'a encore jamais tourné affichera donc une page vide au niveau de son *Overview* : c'est normal, pas une panne.

1. Ouvrez le DAG `example_bash_operator`, puis cliquez sur **Trigger** (en haut à droite). Dans la fenêtre *Trigger Dag* qui s'ouvre, laissez **Single Run** sélectionné et les valeurs par défaut, puis cliquez à nouveau sur **Trigger**.
2. Ouvrez l'onglet **Runs** du DAG : une ligne correspondant à l'exécution que vous venez de lancer doit apparaître.
3. Cliquez sur cette ligne pour ouvrir la vue détaillée du *Dag Run*. Vous y trouverez :
 - l'onglet **Task Instances** : l'état de chaque tâche, sa durée, son nombre de tentatives – c'est l'équivalent de l'ancienne vue *Grid* ;
 - les trois icônes en haut à gauche, qui permettent de basculer vers une représentation en graphe des dépendances – l'équivalent de l'ancienne vue *Graph* ;
 - un clic sur le nom d'une tâche (en bleu, dans *Task Instances*) ouvre son détail, dont l'onglet **Logs**.

3 Créer votre premier DAG

Étape 3

Créez le fichier `dags/pipeline_ventes.py` avec le contenu suivant. Ce squelette définit quatre tâches qui simulent un pipeline de traitement de données de ventes.

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

def extraction_ventes():
    print("Extraction des ventes depuis la source...")

def nettoyage_donnees():
    print("Nettoyage des données...")

def agregation():
    print("Agrégation des ventes par région...")

def chargement_dashboard():
    print("Chargement des résultats vers le dashboard...")
```

```
with DAG(
    dag_id="pipeline_ventes",
    description="Pipeline guidé du TP Jour 1",
    start_date=datetime(2026, 1, 1),
    schedule="@daily",
    catchup=False,
    tags=["tp-jour1"],
) as dag:

    t_extraction = PythonOperator(
        task_id="extraction_ventes",
        python_callable=extraction_ventes,
    )
    t_nettoyage = PythonOperator(
        task_id="nettoyage_donnees",
        python_callable=nettoyage_donnees,
    )
    t_agregation = PythonOperator(
        task_id="agregation",
        python_callable=agregation,
    )
    t_chargement = PythonOperator(
        task_id="chargement_dashboard",
        python_callable=chargement_dashboard,
    )

    t_extraction >> t_nettoyage >> t_agregation >> t_chargement
```

Enregistrez le fichier. Airflow scanne le dossier `dags/` automatiquement (délai d'environ 30 secondes).

Vérification

Rafraîchissez la liste des DAGs dans l'interface : `pipeline_ventes` doit apparaître. Un DAG nouvellement détecté démarre **en pause** (bouton bascule à gauche du nom, désactivé). Cliquez sur **Trigger** : dans la fenêtre *Trigger Dag*, laissez **Single Run** sélectionné et vérifiez que la case “Unpause ... on trigger” est cochée – cela active le DAG et déclenche une exécution en un seul clic. Ouvrez ensuite cette exécution depuis l'onglet **Runs** : les quatre tâches doivent passer à *Success* successivement dans *Task Instances*.

4 Ajouter la gestion des erreurs

Étape 4

Un pipeline réel échoue parfois (source indisponible, données corrompues...). Modifiez la tâche `nettoyage_donnees` pour simuler un échec aléatoire, et ajoutez une politique de reprise.

```
import random
from datetime import timedelta

def nettoyage_donnees():
```

```
if random.random() < 0.6:
    raise Exception("Erreur simulee : donnees source illisibles")
print("Nettoyage des donnees...")
```

Modifiez la définition de la tâche pour ajouter des tentatives automatiques :

```
t_nettoyage = PythonOperator(
    task_id="nettoyage_donnees",
    python_callable=nettoyage_donnees,
    retries=3,
    retry_delay=timedelta(seconds=30),
)
```

Vérification

Déclenchez à nouveau le DAG plusieurs fois jusqu'à obtenir un échec. Depuis l'onglet **Runs**, ouvrez l'exécution concernée, puis cliquez sur la tâche `nettoyage_donnees` dans *Task Instances* : l'onglet **Logs** de la tâche affiche les tentatives successives (*Try 1, Try 2...*) espacées de 30 secondes, avant un succès ou un échec final.

Astuce

Pour observer plusieurs tentatives sans cliquer sur *Trigger* à répétition, vous pouvez temporairement remplacer `schedule="@daily"` par `schedule="* * * * *"` (une exécution par minute). Pensez à repasser à `"@daily"` – ou à mettre le DAG en pause – une fois vos observations terminées, sans quoi Airflow continuera à créer une nouvelle exécution chaque minute indéfiniment.

5 Ajouter une notification d'échec

Étape 5

Dans un contexte réel, une équipe doit être alertée en cas d'échec définitif (email, Slack...). Pour ce TP, vous allez simuler cette alerte en écrivant dans un fichier de log.

```
def notifier_echec(context):
    with open("/opt/airflow/logs/notifications.log", "a") as f:
        f.write(
            f"ECHEC - dag={context['dag'].dag_id} "
            f"task={context['task_instance'].task_id} "
            f"date={context['logical_date']}\n"
        )
```

Rattachez cette fonction à la tâche `nettoyage_donnees` :

```
t_nettoyage = PythonOperator(
    task_id="nettoyage_donnees",
    python_callable=nettoyage_donnees,
    retries=3,
    retry_delay=timedelta(seconds=30),
    on_failure_callback=notifier_echec,
```

```
)
```

Astuce

La fonction n'est appelée qu'après l'échec de **toutes** les tentatives (les 3 *retries* épuisés), pas à chaque tentative individuelle. Notez aussi la clé `logical_date` (et non `execution_date`) : Airflow 3 a retiré `execution_date` du contexte des tâches – l'utiliser lèverait une `KeyError` et empêcherait le callback de s'exécuter.

Vérification

Déclenchez le DAG jusqu'à provoquer un échec définitif. Le fichier est directement accessible sur votre machine hôte (le `docker-compose.yaml` officiel monte `./logs` en volume) : depuis PowerShell, dans le dossier `tp-airflow`, exécutez `Get-Content .\logs\notifications.log`. Une ligne doit être apparue avec l'identifiant du DAG, de la tâche et la date d'exécution.

Astuce

Avec l'exécuteur Celery utilisé par Airflow 3 (services `airflow-worker`, `airflow-scheduler`, `airflow-dag-processor` séparés), le callback s'exécute en réalité dans le conteneur `airflow-worker`. Si vous préférez consulter le fichier depuis l'intérieur d'un conteneur plutôt que sur l'hôte : `docker compose exec airflow-worker cat /opt/airflow/logs/notifications.log`.

6 Vérifier la traçabilité

Étape 6

Terminez en explorant l'historique complet de vos exécutions :

1. Dans l'onglet **Runs** du DAG, observez les différentes exécutions et leur état (*Success*, *Failed*, *Running*).
2. Cliquez sur une exécution pour ouvrir *Task Instances* : c'est l'équivalent de l'ancienne vue *Grid*, avec le détail par tâche (durée, tentatives, statut).
3. Cliquez sur le nom d'une tâche pour ouvrir son détail complet, dont les logs.
4. Notez comment cet historique permettrait de répondre à un auditeur qui demanderait « le pipeline de ventes a-t-il tourné correctement le 15 du mois dernier ? ».

Livrable attendu

Prenez une capture d'écran de la page *Task Instances* d'une exécution montrant au moins un succès et un échec, et joignez le contenu de `notifications.log` à votre rendu.

7 Questions de compréhension

1. Quelle est la différence entre une *task* et un *operator* dans Airflow ?

2. Pourquoi le DAG doit-il être acyclique (sans boucle) ?
3. Que se passe-t-il si `retries=3` et que la tâche échoue à chaque tentative ? Combien de fois la fonction est-elle exécutée au total ?
4. Pourquoi `on_failure_callback` n'est-il déclenché qu'après épuisement des tentatives, et pas à chaque essai ?
5. En quoi l'historique conservé par Airflow contribue-t-il à la conformité et à l'auditabilité d'un traitement de données ?

Pour aller plus loin (optionnel)

- Remplacez `schedule="@daily"` par une expression cron ("`0 6 * * 1-5`") pour ne s'exécuter que les jours ouvrés à 6h.
- Testez un intervalle très court ("`* * * * *`", toutes les minutes) pour observer rapidement le comportement du *scheduler* et des *retries* en conditions réelles. Avec `catchup=False`, Airflow ne rattrape que les intervalles à venir, pas l'historique depuis `start_date` – sans quoi des milliers d'exécutions se déclencheraient d'un coup. Pensez à mettre le DAG en pause une fois l'observation terminée.
- Ajoutez une cinquième tâche qui ne se déclenche que si `chargement_dashboard` réussit, en utilisant `trigger_rule`.
- Explorez les *Sensors* Airflow, qui attendent qu'une condition externe soit remplie (arrivée d'un fichier, réponse d'une API) avant de démarrer une tâche.